

Data di consegna: 06/01/2026

# Progetto Assembly RISC-V per il Corso di Architetture degli Elaboratori – A.A. 2024/2025 –

## Gestione di Liste Concatenate in Assembly Risc-V

Il presente progetto consiste nell'implementazione in linguaggio *Assembly RISC-V* di una struttura dati di tipo **lista concatenata** (*Linked List*). L'obiettivo principale è la **gestione dinamica** dei nodi attraverso l'elaborazione di una *stringa di input* contenente una sequenza di comandi operativi, permettendo la manipolazione e l'organizzazione dei dati in memoria a basso livello.

Le operazioni richieste di poter gestire sono le seguenti (formattazione inclusa):

**ADD(x);** -aggiunge un nodo con dato *x* in coda alla struttura dati.

**DEL(x);** -elimina ogni nodo che contenga il dato *x* dalla struttura.

**PRINT;** -stampa la lista partendo dalla testa della lista.

**SORT;** -ordina la lista.

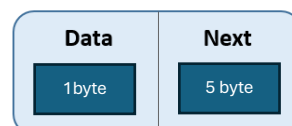
**REV.** -inverte l'ordine degli elementi nella lista.

In questa relazione vedremo il metodo che ho adottato per la loro implementazione ed eventuali funzioni di supporto che ho utilizzato.

Come imposto dalle linee guida del progetto, ogni nodo della lista è così composto:

1 byte per il dato;

4 byte per il puntatore al nodo successivo.



### Metodo di implementazione

Sezione `.data`:

Definisce '**listinput**' su cui il codice opererà

Sezione `.text`:

```
32 la s1,listInput # stringa
33 li s2,0 # testa
34 li s3,0 # coda
35 li s4,0 # numero nodi
36 li s5,0x20000000 # prossimo indirizzo per l'inserimento dei nodi
37 jal parse_listinput
```

Inizio definendo i *Saved Registers* di cui il programma fruirà durante tutta la sua esecuzione:

Data di consegna: 06/01/2026

- **S1**: l'indirizzo al primo byte di listinput;
- **S2**: l'indirizzo della testa (inizialmente vuota) della lista;
- **S3**: l'indirizzo della coda (inizialmente coincidente con la testa);
- **S4**: un contatore dei nodi presenti nella lista;
- **S5**: rappresenta il prossimo indirizzo di memoria libero per la scrittura di un nodo.

## Modulo di Parsing e Validazione

La procedura di **parsing** **analizza la stringa** di input ignorando spazi e separatori. Per ogni comando individuato (ADD, DEL, PRINT, SORT, REV), il sistema esegue una **validazione sintattica rigorosa** (es. controllo delle parentesi e del range ASCII del parametro). In caso di formato non valido, il programma attiva una routine di recupero che **scarta** il comando errato e prosegue fino alla successiva tilde.

### Struttura del parsing:

1. **Legge** 1 byte in *s1*;
2. **Salta** eventuali **spazi** e **tilde** ripetuti (*skip\_char*)
3. Se trova **'\0'** **termina** l'esecuzione del programma (*end\_parse*);
4. **Cerca** una **corrispondenza** tra il primo **carattere** trovato ed il primo carattere di ogni comando (*search\_command*);
5. **Trovata** una corrispondenza controlla che il **formato** sia **valido** (es: ~ADD(x)~ e non ADD(x)~);
  - a. Nel caso di ADD(x) e DEL(x) legge e **si salva il parametro** tra parentesi nel registro *a0*.
  - b. In tutti i casi **controlla** che il comando inizi con una **tilde** e **termini** con una **tilde** (*eccetto il primo e l'ultimo comando della lista*).
6. **Verificata** la validità del formato, la funzione di *parsing* **passa il comando** alla funzione corrispondente tramite **un salto condizionale** che, una volta tornato all'*ra* salterà a *next\_command* che, prima di cercare una nuova corrispondenza con un nuovo comando, si assicura di **allineare s1 ad una tilde o un carattere**.
7. Il programma è progettato per essere **resiliente a input malformati**: Se un comando non rispetta il formato (es. DEL(bb) o PRINT scritto male), la procedura *invalid\_format* ne interrompe l'esecuzione e il parser salta a *next\_command*.

Inoltre, viene verificato che ogni parametro inserito rientri nel range ASCII valido [32, 125].

```

52 parsing_loop:
53     lb t0,0(s1) #carico il carattere corrente
54     beqz t0,end_parse
55
56     li t1,32 #spazio
57     beq t0,t1,skip_char
58
59     li t1,126 #tilde
60     beq t0,t1,skip_char
61
62     jal search_command
63
64
65 skip_char:
66     addi s1,s1,1
67     j parsing_loop
68
69 end_parse:
70     lw ra,0(sp)
71     addi sp,sp,4
72     ret

```

Figura 1 base del parsing

```

search_command:
    addi sp,sp,-4
    sw ra,0(sp)

    lb t0,0(s1) # carica carattere corrente

    li t1,65 #A
    beq t0,t1,add_command

    li t1,68 #D
    beq t0,t1,del_command

    li t1,82 #R
    beq t0,t1,rev_command

    li t1,80 #P
    beq t0,t1,print_command

    li t1,83 #S
    beq t0,t1,sort_command

    j invalid_command

```

Figura 2 ricerca del match del carattere

```

97 next_command:
98     #trova tilde / stringa#
99     lb t0,0(s1)
100     beqz t0,end_parse
101
102     li t1,126
103     beq t0,t1,skip_char
104     addi s1,s1,1
105     j next_command

```

Figura 3 ricerca del prossimo comando

## Operazione 'ADD':

Ho strutturato l'operazione di aggiunta di un nodo nelle seguenti fasi:

1. Come prima azione è fondamentale **ricercare il prossimo indirizzo libero** per la scrittura. Per fare ciò ADD si avvale di una funzione di supporto *'find\_next\_free\_addr'*. La funzione *find\_next\_free\_addr* implementa una **gestione dinamica** della memoria ricercando il primo spazio contiguo **di 5 byte** disponibile. Questo meccanismo ottimizza l'uso

Data di consegna: 06/01/2026

delle risorse permettendo il riutilizzo delle locazioni rilasciate e riducendo gli sprechi dell'allocazione statica.

2. Una volta individuato l'indirizzo, la procedura vi memorizza il dato nel primo byte.
3. Se la lista era **vuota**, il nodo aggiunto diventa sia testa (s4) che coda(s3) della struttura e **termina** l'operazione di aggiunta, altrimenti:
  - **Aggiorna** il puntatore del nodo **precedente** (la coda);
  - **Aggiorna** la **coda** (s3) che diventa ora il nodo appena aggiunto;
4. **Incremento** il contatore dei nodi (**s4++**).

## Operazione 'DEL':

La procedura di eliminazione di un nodo scorre tutta la lista e per ogni nodo ne controlla il dato.

Se il dato corrisponde al parametro con cui è stata chiamata la procedura, allora si procede con l'eliminazione del nodo seguendo questi passi:

1. Se la lista è vuota, l'operazione **termina subito**.
2. (*check\_head\_deletion*): se il nodo è la testa, la testa diventa il nodo successivo (o nulla se il counter dei nodi s4 è 1, in tal caso aggiornerò la coda di conseguenza);
3. Se il dato nel nodo **non corrisponde** al parametro di ricerca, **salvo l'indirizzo** del nodo e passo a controllare il prossimo;
4. In caso di eliminazione **al centro** della lista, la procedura non fa altro che **aggiornare il puntatore del precedente** nodo in modo che punti al nodo successivo a quello eliminato;
5. Per ogni eliminazione che avviene **decremento** il contatore dei nodi
6. DEL termina quando raggiunge la fine della lista;

## Operazione 'PRINT':

Questa è una procedura che viene gestita ricorsivamente. Ad ogni chiamata accede al dato del nodo che gli viene passato come parametro in a0 e lo stampa tramite le istruzioni in *fig. 1.p*

1. Ad ogni ricorsione, viene caricato il puntatore al nodo successivo e passato alla chiamata successiva.
2. Lo stack di chiamate termina non appena il nodo passato è nullo; a quel punto, tornando al chiamante iniziale, viene ripristinata la memoria dello stack a ritroso.

Data di consegna: 06/01/2026

```
505 lb a0,0(a0) #  
506 li a7,11  
507 ecall
```

Figura 4

```
print_recursive:  
    beqz a0, return_print # caso base testa = null  
  
    addi sp,sp,-8 # salvo a0 e ra nello stack  
    sw ra,0(sp)  
    sw a0,4(sp)
```

Figura 2 inizio ricorsione

```
lw ra,0(sp)  
addi sp,sp,8 # ripristino la stack  
  
return_print:  
    ret
```

Figura 3 ripristino stack e return al chiamante

## Operazione 'SORT':

Per l'operazione di ordinamento ho scelto un algoritmo di *Bubble Sorting*. Inoltre, ogni scambio tra nodi non è implementato come un effettivo 'swap' dei nodi ma mi sono limitata a scambiare solamente il 'data' dei nodi stessi. La scelta della tipologia di sorting è stata ponderata sui seguenti vantaggi:

- **Efficienza su Dataset Ridotti:** Nonostante la complessità teorica  $O(n^2)$ , su un massimo di 30 elementi il tempo di esecuzione è trascurabile e competitivo con algoritmi più complessi.
- **Iteratività:** L'implementazione iterativa evita l'uso intensivo dello stack e la gestione di ricorsioni profonde.
- **Ottimizzazione dello Swap:** Risulta più efficiente scambiare il singolo byte del campo data piuttosto che manipolare i 4 byte dei puntatori, operazione che richiederebbe una logica di aggiornamento dei collegamenti molto più dispendiosa in Assembly.

Funzionamento del sort:

1. **Inizializzazione e Controllo Lista:** La procedura verifica se la lista è vuota o contiene un solo elemento tramite il registro s4 (numero nodi); in tal caso, termina immediatamente.
2. **Ciclo Esterno:** Viene utilizzato un ciclo principale (*outer\_sort*) che controlla il numero di scorrimenti totali sulla lista. Un registro flag (t2) viene azzerato all'inizio di ogni scorrimento per monitorare se avvengono scambi; se alla fine di uno scorrimento non ci sono stati scambi, l'algoritmo termina anticipatamente (*ottimizzazione*).
3. **Ciclo Interno (Confronto tra adiacenti):** Attraverso *inner\_sort*, il programma scorre la lista nodo per nodo. Per ogni coppia di nodi adiacenti, vengono caricati i rispettivi valori data per il confronto.
4. **Determina categoria carattere:** viene chiamata la funzione *get\_category* per assegnare un peso gerarchico ai caratteri. Il confronto avviene prima sulla categoria e, in caso di parità, sul valore ASCII effettivo per garantire un ordine crescente totale.
5. **Scambio Dati (Swap):** Se l'ordine dei due nodi è errato, i valori dei registri temporanei vengono invertiti e riscritti nelle rispettive locazioni di memoria. Viene effettuato lo swap come sopra spiegato.

Data di consegna: 06/01/2026

## Operazione 'REV':

Questa procedura sfrutta lo stack di sistema per invertire l'ordine degli elementi, preservando la struttura della lista ma riorganizzando i valori in modo efficiente.

1. **rev\_push\_phase:** Partendo dalla testa della lista, la scorriamo tutta e parte una fase di 'push' dei dati nello stack che, alla fine dello scorrimento, conterrà i dati di tutti i nodi.
2. **Rev\_pop\_phase:** adesso che tutti i dati dei nodi sono nello stack, per recuperarli in ordine inverso, alla procedura non basta che pescarli uno alla volta dallo stack; assegnarli a partire dal primo nodo; passare al nodo successivo e ripetere l'iterazione finché non saranno estratti tutti i dati.

```

542 rev_push_phase:
543 beqz t1, end_rev_push_phase # se il prossimo nodo e' nullo, termina push
544 lb t3, 0(t1) # dato del nodo corrente
545 addi sp, sp, -4 # evito problemi di allineamento
546 sb t3, 0(sp) # push del dato del nodo corrente nello stack
547
548 addi t2, t2, 1 # aumento contatore pushati
549 addi t4, t1, 1 # allineo al puntatore
550
551 lw t1, 0(t4) # carico l'indirizzo al successivo
552 j rev_push_phase

```

Figura 5 fase di push

```

557 rev_pop_phase:
558 beqz t2, end_rev # se il contatore ? a 0, ho finito
559
560 lb t3, 0(sp) # pop
561 sb t3, 0(t1) # scrivo il data popped nel nodo corrente
562 addi sp, sp, 4
563 addi t2, t2, -1
564
565 addi t4, t1, 1 #allineo al puntatore
566 lw t1, 0(t4) # carico il nodo successivo e continuo
567 j rev_pop_phase

```

Figura 6 fase di pop

Vantaggi nella scelta dell'uso diretto dello stack:

1. **Integrità Strutturale:** Non modificando i puntatori NEXT, non c'è alcun rischio di "rompere" la catena della lista o creare loop infiniti. La struttura fisica della lista in memoria rimane identica; cambiano solo i dati contenuti.
2. **Semplicità del Codice:** L'uso dello stack elimina la necessità di gestire tre puntatori contemporaneamente (precedente, corrente, successivo), riducendo il numero di registri temporanei necessari e la complessità del controllo di flusso.

## Test

Oltre alle immagini sottostanti, è possibile visionare la registrazione schermo durante l'esecuzione dei test.

### Test 1:

listinput = "ADD(1) ~ ADD(a) ~ add(B) ~ ADD(B) ~ ADD ~ ADD(9) ~PRINT~SORT(a)~PRINT~DEL(bb) ~DEL(B) ~PRINT~REV~PRINT"

Output atteso: 1 a B 9      1 a B 9      1 a 9      9 a 1

Prima dell'esecuzione:

The screenshot shows a debugger interface with two main panels. The left panel displays assembly code for a program named 'test'. The code includes comments in Italian and assembly instructions for parsing a string, allocating memory, and performing operations like ADD, PRINT, SORT, and REV. The right panel shows a register window with columns for Name, Alias, and Value. The registers x0 through x19 are listed, with x0 containing 0, x1 containing 0, x2 containing 27147483632, and x3 containing 268435456. The registers x4 through x19 are all 0.

Data di consegna: 06/01/2026

Dopo l'esecuzione:

The screenshot shows a debugger window with the following components:

- Source code:** Assembly code for a program that calculates the sum of elements in an array and prints the result. The code includes comments in Italian and assembly instructions like `.string`, `.add`, `.del`, `.print`, `.sort`, `.rev`, and `.print`.
- Console:** Displays the output of the program: `1 a B 9`, `1 a B 9`, `1 a 9`, and `9 a 1`. Below the output, it says "Program exited with code: 0".
- GPR:** A table of General Purpose Registers (GPR) showing their values after execution. The table has columns for Name, Alias, and Value.

| Name | Alias | Value      |
|------|-------|------------|
| x1   | ra    | 28         |
| x2   | sp    | 2147483632 |
| x3   | gp    | 268435456  |
| x4   | tp    | 0          |
| x5   | t0    | 0          |
| x6   | t1    | 32         |
| x7   | t2    | 0          |
| x8   | s0    | 0          |
| x9   | s1    | 268435572  |
| x10  | a0    | 10         |
| x11  | a1    | 1          |
| x12  | a2    | 0          |
| x13  | a3    | 0          |
| x14  | a4    | 0          |
| x15  | a5    | 0          |
| x16  | a6    | 0          |
| x17  | a7    | 10         |
| x18  | s2    | 536870912  |
| x19  | s3    | 536870927  |
| x20  | s4    | 3          |
| x21  | s5    | 536870932  |

Si nota che sono state eseguite con successo le 4 operazioni di print, le quali ci mostrano anche il successo delle operazioni antecedenti ad esse. Inoltre, il valore dello stack (sp) evidenziato in alto a destra, è stato ristabilito a quello che era il suo valore prima dell'esecuzione. Ciò denota il corretto ripristino dello stack.

## Test 2:

listinput = "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(;) ~ ~PRI~REV~PRINT"

Output atteso: \_\_\_\_\_ ; B a a 1

Prima dell'esecuzione:

The screenshot shows a debugger window with the following components:

- Source code:** Assembly code for a program that calculates the sum of elements in an array and prints the result. The code includes comments in Italian and assembly instructions like `.string`, `.add`, `.del`, `.print`, `.sort`, `.rev`, and `.print`.
- Console:** Displays the output of the program: `1 a B 9`, `1 a B 9`, `1 a 9`, and `9 a 1`. Below the output, it says "Program exited with code: 0".
- GPR:** A table of General Purpose Registers (GPR) showing their values before execution. The table has columns for Name, Alias, and Value.

| Name | Alias | Value      |
|------|-------|------------|
| x0   | zero  | 0          |
| x1   | ra    | 0          |
| x2   | sp    | 2147483632 |
| x3   | gp    | 268435456  |
| x4   | tp    | 0          |
| x5   | t0    | 0          |
| x6   | t1    | 0          |
| x7   | t2    | 0          |
| x8   | s0    | 0          |
| x9   | s1    | 0          |
| x10  | a0    | 0          |
| x11  | a1    | 0          |
| x12  | a2    | 0          |
| x13  | a3    | 0          |
| x14  | a4    | 0          |
| x15  | a5    | 0          |
| x16  | a6    | 0          |
| x17  | a7    | 0          |
| x18  | s2    | 0          |
| x19  | s3    | 0          |
| x20  | s4    | 0          |

Data di consegna: 06/01/2026

Dopo l'esecuzione:

Assembly code snippet:

```

20 #listInput: .string "ADD(A)-ADD(B)-ADD(C)-REV-REV-PRINT" #doppia inversione
21 #listInput: .string "ADD(C)-ADD(A)-ADD(B)-SORT-REV-PRINT" #sort + rev = ordine decrescente
22
23 #ESEMPI NELLA RELAZIONE
24 #listInput: .string "ADD(1) ~ ADD(a) ~ add(B) ~ ADD(B) ~ ADD ~ ADD(9) ~PRINT-SORT(a)-PRINT-DEL(bb) ~DEL(B) ~PRINT-REV-PRINT"
25 #listInput: .string "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(:) ~PRI-REV-PRINT"
26 #listInput: .string "ADD(:) ~ ADD(f) ~ ADD (a) ~ADD(1) ~ADD(A) ~ADD(b) ~ ~ del(:) ~PRINT-SORT-REV-PRINT ~ DEL(f)-Print~ PRINT~ REV PRINT"
27
28 .text
29 la s1,listInput # stringa
30 li s2,0 # testa
31 li s3,0 # coda
32 li s4,0 # numero nodi
33 li s5,0x20000000 # prossimo indirizzo per l'inserimento dei nodi
34 jal parse_listinput
35
36 #fine programma
37 li a7,10
38 ecall
39
40 ##### main #####
41 #
42 #####

```

Register window values:

| Name | Alias | Value      |
|------|-------|------------|
| x0   | zero  | 0          |
| x1   | ra    | 28         |
| x2   | sp    | 2147483632 |
| x3   | gp    | 268435456  |
| x4   | tp    | 0          |
| x5   | t0    | 0          |
| x6   | t1    | 32         |
| x7   | t2    | 0          |
| x8   | s0    | 0          |
| x9   | s1    | 268435522  |
| x10  | a0    | 10         |
| x11  | a1    | 1          |
| x12  | a2    | 0          |
| x13  | a3    | 0          |
| x14  | a4    | 0          |
| x15  | a5    | 0          |
| x16  | a6    | 0          |
| x17  | a7    | 10         |
| x18  | s2    | 536870912  |
| x19  | s3    | 536870932  |
| x20  | s4    | 5          |

Console output: B a a 1

Program exited with code: 0

È stata eseguita una sola print, come atteso. Inoltre, l'operazione di REV è andata a buon fine. Anche in questo caso lo stack è stato ripristinato correttamente.

## Test 3:

listinput = "ADD(:) ~ ADD(f) ~ ADD (a) ~ADD(1) ~ADD(A) ~ADD(b) ~ ~ del(:) ~PRINT~  
SORT~REV~PRINT ~ DEL(f)~Print~ PRINT~ REV PRINT"

Output atteso: : f 1 A b A f b 1 : A b 1 :

Prima dell'esecuzione:

Assembly code snippet:

```

20 #listInput: .string "ADD(A)-ADD(B)-ADD(C)-REV-REV-PRINT" #doppia inversione
21 #listInput: .string "ADD(C)-ADD(A)-ADD(B)-SORT-REV-PRINT" #sort + rev = ordine decrescente
22
23 #ESEMPI NELLA RELAZIONE
24 #listInput: .string "ADD(1) ~ ADD(a) ~ add(B) ~ ADD(B) ~ ADD ~ ADD(9) ~PRINT-SORT(a)-PRINT-DEL(bb) ~DEL(B) ~PRINT-REV-PRINT"
25 #listInput: .string "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(:) ~PRI-REV-PRINT"
26 #listInput: .string "ADD(:) ~ ADD(f) ~ ADD (a) ~ADD(1) ~ADD(A) ~ADD(b) ~ ~ del(:) ~PRINT-SORT-REV-PRINT ~ DEL(f)-Print~ PRINT~ REV PRINT"
27
28 .text
29 la s1,listInput # stringa
30 li s2,0 # testa
31 li s3,0 # coda
32 li s4,0 # numero nodi
33 li s5,0x20000000 # prossimo indirizzo per l'inserimento dei nodi
34 jal parse_listinput
35
36 #fine programma
37 li a7,10
38 ecall
39
40 ##### main #####
41 #
42 #####

```

Register window values:

| Name | Alias | Value      |
|------|-------|------------|
| x0   | zero  | 0          |
| x1   | ra    | 0          |
| x2   | sp    | 2147483632 |
| x3   | gp    | 268435456  |
| x4   | tp    | 0          |
| x5   | t0    | 0          |
| x6   | t1    | 0          |
| x7   | t2    | 0          |
| x8   | s0    | 0          |
| x9   | s1    | 0          |
| x10  | a0    | 0          |
| x11  | a1    | 0          |
| x12  | a2    | 0          |
| x13  | a3    | 0          |
| x14  | a4    | 0          |
| x15  | a5    | 0          |
| x16  | a6    | 0          |
| x17  | a7    | 0          |
| x18  | s2    | 0          |
| x19  | s3    | 0          |
| x20  | s4    | 0          |

Console output: (empty)

Data di consegna: 06/01/2026

Dopo l'esecuzione:

```
20 #listInput: .string "ADD(A)-ADD(B)-ADD(C)-REV-REV-PRINT" #doppia inversione
21 #listInput: .string "ADD(C)-ADD(A)-ADD(B)-SORT-REV-PRINT" #sort + rev = ordine decrescente
22
23 #ESEMPI NELLA RELAZIONE
24 #listInput: .string "ADD(1) ~ ADD(a) ~ add(B) ~ ADD(B) ~ ADD ~ ADD(9) ~PRINT-SORT(a)-PRINT-DEL(bb) ~DEL(B) ~PRINT-REV-PRINT"
25 #listInput: .string "ADD(1) ~ ADD(a) ~ ADD(a) ~ ADD(B) ~ ADD(:) ~PRI-REV-PRINT"
26 listInput: .string "ADD(:) ~ ADD(f) ~ ADD (a) ~ADD(1) ~ADD(A) ~ADD(b) ~ ~ del(:) ~PRINT-SORT-REV-PRINT ~ DEL(f)-Print~ PRINT~ REV PRINT"
27
28 .text
29 la s1,listInput # stringa
30 li s2,0 # testa
31 li s3,0 # coda
32 li s4,0 # numero nodi
33 li s5,0x20000000 # prossimo indirizzo per l'inserimento dei nodi
34 jal parse_listinput
35
36 #fine programma
37 li a7,10
38 ecall
39
40 #####
41 # main #
42 #####
```

Console

```
: f 1 A b
A f b 1 :
A b 1 :
```

Program exited with code: 0

| Name | Alias | Value      |
|------|-------|------------|
| x0   | zero  | 0          |
| x1   | ra    | 28         |
| x2   | sp    | 2147483632 |
| x3   | gp    | 268435456  |
| x4   | tp    | 0          |
| x5   | t0    | 0          |
| x6   | t1    | 126        |
| x7   | t2    | 536870932  |
| x8   | s0    | 0          |
| x9   | s1    | 268435587  |
| x10  | a0    | 10         |
| x11  | a1    | 0          |
| x12  | a2    | 0          |
| x13  | a3    | 0          |
| x14  | a4    | 0          |
| x15  | a5    | 0          |
| x16  | a6    | 0          |
| x17  | a7    | 10         |
| x18  | s2    | 536870912  |
| x19  | s3    | 536870932  |
| x20  | s4    | 4          |

Display type: Signed

Ed anche in questo terzo ed ultimo test tutte le operazioni sono eseguite in maniera corretta ed i comandi mal formattati sono stati correttamente scartati.

## Conclusioni

In conclusione, l'analisi dei risultati ottenuti e i test approfonditi effettuati confermano la validità e la robustezza della soluzione implementata. In particolare si nota:

- **Resilienza agli input:** Il modulo di parsing identifica e neutralizza correttamente i comandi malformati, garantendo la continuità dell'esecuzione senza compromettere lo stato della lista.
- **Integrità della struttura:** Le strategie di inserimento (ADD) ed eliminazione (DEL) preservano la coerenza dei puntatori e dei riferimenti testa-coda, mantenendo la logica della lista concatenata sempre valida.
- **Efficienza e gestione della memoria:** La funzione di allocazione dinamica manuale ottimizza l'uso dello spazio, mentre il rigoroso bilanciamento dello stack in ogni procedura assicura la stabilità del sistema, prevenendo fenomeni di *stack leak* o *overflow*.

In sintesi, il progetto sembrerebbe rispondere ai requisiti tecnici richiesti.